



PRÁCTICA 4. HERRAMIENTAS PARA ADMINISTRACIÓN DE LOGS. PARTE 3

Valeria Hernández Cortés

Grupo IDGS17

Consultas con LogQL y Construcción de Dashboards para el Análisis de Registros

Índice

Consultas con LogQL y Construcción de Dashboards para el Análisis de Registros...	1
Índice.....	1
Objetivo.....	2
Introducción.....	3
Sintaxis básica de LogQL.....	4
Desarrollo de la práctica.....	5
Paso 1. Verificación de la plataforma.....	5
Paso 2. Exploración de la interfaz Explore.....	6
Paso 3. Construcción de consultas básicas.....	6
Paso 4. Construcción de consultas específicas.....	7
Paso 5. Generación de actividad.....	9
Paso 6. Análisis de un incidente.....	10
Paso 7. Construcción del dashboard.....	11
Comparación entre Seq y Grafana + Loki.....	12
Caso de estudio.....	13
Pregunta de reflexión: ¿Qué ventajas aporta LogQL frente a revisar directamente archivos de texto o realizar búsquedas simples en Seq?.....	14
Conclusiones.....	15

Objetivo

Utilizar Grafana y el lenguaje de consultas LogQL para consultar, filtrar y analizar los registros generados por VulnerableApp. Asimismo, construir dashboards que faciliten el monitoreo de la aplicación y apoyen la investigación de incidentes mediante el análisis de los registros almacenados en Loki.

Introducción

En las prácticas anteriores se configuró una plataforma de observabilidad utilizando Serilog, Promtail, Loki y Grafana. Gracias a esta infraestructura, los registros generados por VulnerableApp se envían automáticamente tanto a Seq como a Loki para su almacenamiento y análisis.

En esta práctica el objetivo ya no es modificar la infraestructura, sino aprovechar la información almacenada mediante consultas LogQL. Para ello se utilizan las labels configuradas en Promtail, las cuales clasifican automáticamente los registros de seguridad, auditoría y errores de la aplicación. Posteriormente, esta información se organiza en dashboards que permiten supervisar el comportamiento del sistema y facilitar la detección de incidentes.

Sintaxis básica de LogQL

Antes de comenzar con las consultas, se revisó la sintaxis básica de LogQL para comprender cómo localizar información específica dentro de los registros almacenados en Loki. En esta práctica se aprovecharon las labels configuradas previamente en Promtail para clasificar automáticamente los eventos según su tipo.

Consulta	Propósito	Resultado esperado
<code>{job="vulnerableapp"}</code>	Mostrar todos los registros generados por VulnerableApp.	Se visualizan todos los eventos enviados por la aplicación.
<code>{Security!=""}</code>	Mostrar únicamente los registros clasificados como eventos de seguridad.	Se observan autenticaciones, credenciales inválidas y otros eventos relacionados con la seguridad.
<code>{Audit!=""}</code>	Consultar únicamente los eventos de auditoría generados por la aplicación.	Se muestran registros informativos asociados a las acciones realizadas por los usuarios.
<code>`{job="vulnerableapp"}`</code>	<code>= "Error"</code>	Buscar mensajes que contengan la palabra Error.
<code>`{job="vulnerableapp"}`</code>	<code>= "Warning"</code>	Filtrar mensajes que contienen advertencias.
<code>`{Security!=""}`</code>	<code>= "hacker"</code>	Combinar una label con una búsqueda de texto.

Desarrollo de la práctica

Paso 1. Verificación de la plataforma

Antes de comenzar las consultas se verificó que todos los componentes de la plataforma continuaran funcionando correctamente. Para ello se ejecutó el comando `docker ps`, comprobando que los contenedores de Grafana, Loki y Promtail se encontraban activos.

Posteriormente se inició `VulnerableApp` y se realizaron distintas acciones, como inicios de sesión, consultas y operaciones dentro de la aplicación, con el propósito de generar nuevos registros. Finalmente, se verificó que tanto Seq como Grafana recibieran la información correctamente.

```
C:\Users\valec>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8b3d1e3b0420	grafana/grafana:latest	"/run.sh"	4 days ago	Up About a minute	0.0.0.0:3000->3000/tcp	grafana
5341b91f9dd5	grafana/promtail:latest	"/usr/bin/promtail -c..."	4 days ago	Up About a minute		promtail
827fc4953c51	grafana/loki:latest	"/usr/bin/loki -conf..."	4 days ago	Up About a minute	0.0.0.0:3100->3100/tcp	loki
a142a0e5e516	datast/seq	"/bin/seqentry"	13 days ago	Up About a minute	443/tcp, 5341/tcp, 45341/tcp, 0.0.0.0:5341->80/tcp	seq

Figura 1. Contenedores de la plataforma en ejecución.

The screenshot displays the Seq application interface. The main window shows a list of log events with columns for time, level, and message. The sidebar on the right contains sections for 'SIGNALS', 'QUERIES', and 'VARIABLES'. The 'SIGNALS' section shows a list of signals with their levels and counts. The 'QUERIES' section shows a list of queries with their names and counts. The 'VARIABLES' section shows a list of variables with their names and counts. The bottom of the sidebar shows the license information: '2025.2.16202 Individual License'.

Figura 2. Recepción de registros en Seq.

Paso 2. Exploración de la interfaz Explore

Una vez confirmado el funcionamiento de la plataforma, se accedió a la sección Explore de Grafana utilizando Loki como fuente de datos.

Durante la exploración se identificaron los siguientes elementos principales:

1. Editor de consultas: espacio donde se escriben las consultas LogQL para filtrar la información almacenada en Loki.
2. Selector de tiempo: permite definir el intervalo de tiempo sobre el cual se realizará la búsqueda de registros.
3. Panel de resultados: muestra una gráfica con la cantidad de eventos encontrados y la lista completa de registros que cumplen con la consulta.

Esta interfaz facilita realizar consultas de forma interactiva y analizar rápidamente el comportamiento de la aplicación.

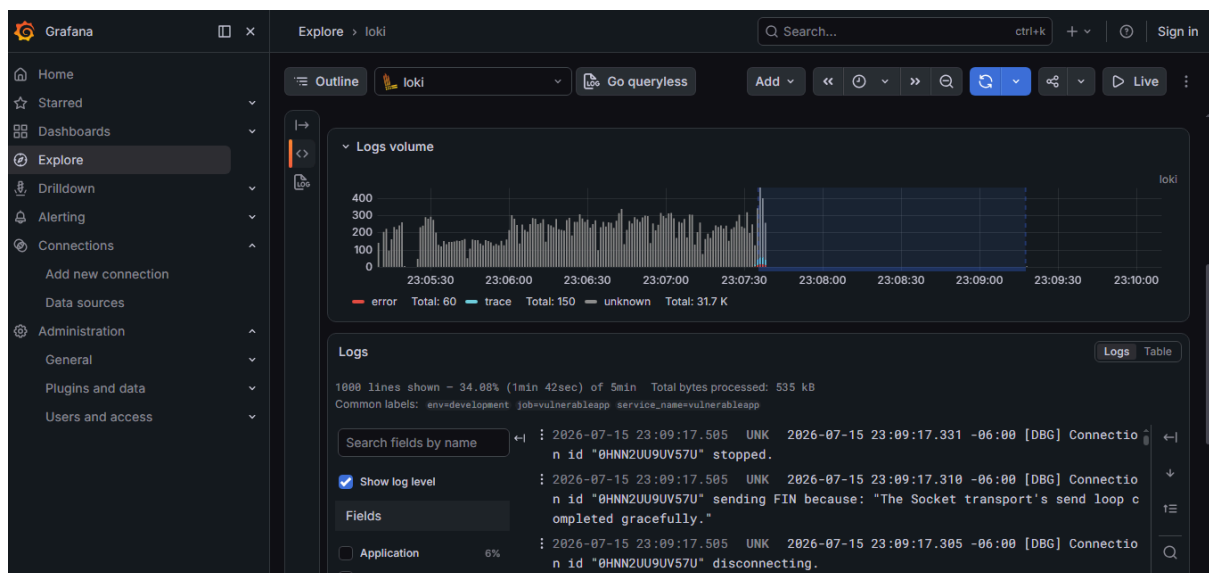


Figura 3. Interfaz Explore de Grafana.

Paso 3. Construcción de consultas básicas

Una vez familiarizados con la interfaz, se realizaron las consultas básicas utilizando las labels configuradas en Promtail.

Las consultas permitieron recuperar distintos tipos de registros:

1. Eventos generales de la aplicación mediante la label job.
2. Eventos de seguridad mediante la label Security.
3. Eventos de auditoría mediante la label Audit.

4. Registros que contenían errores y advertencias utilizando filtros por contenido.

Cada consulta devolvió únicamente los registros que cumplían con los criterios establecidos, lo que facilitó localizar información específica sin necesidad de revisar todos los eventos almacenados.

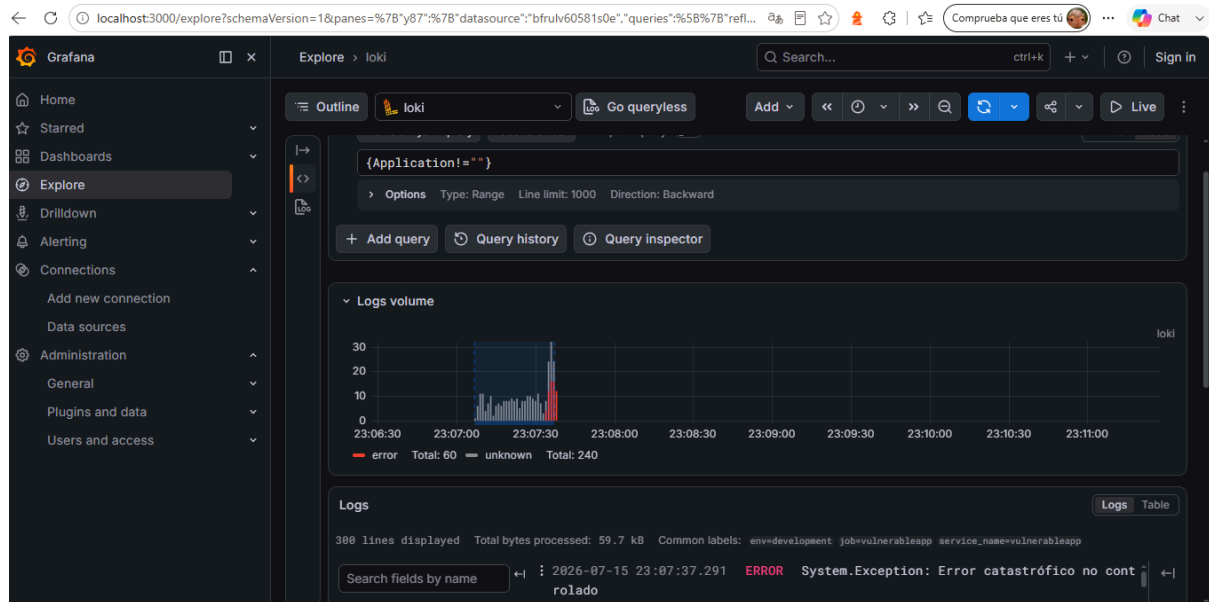


Figura 5. Consulta de eventos de Errores.

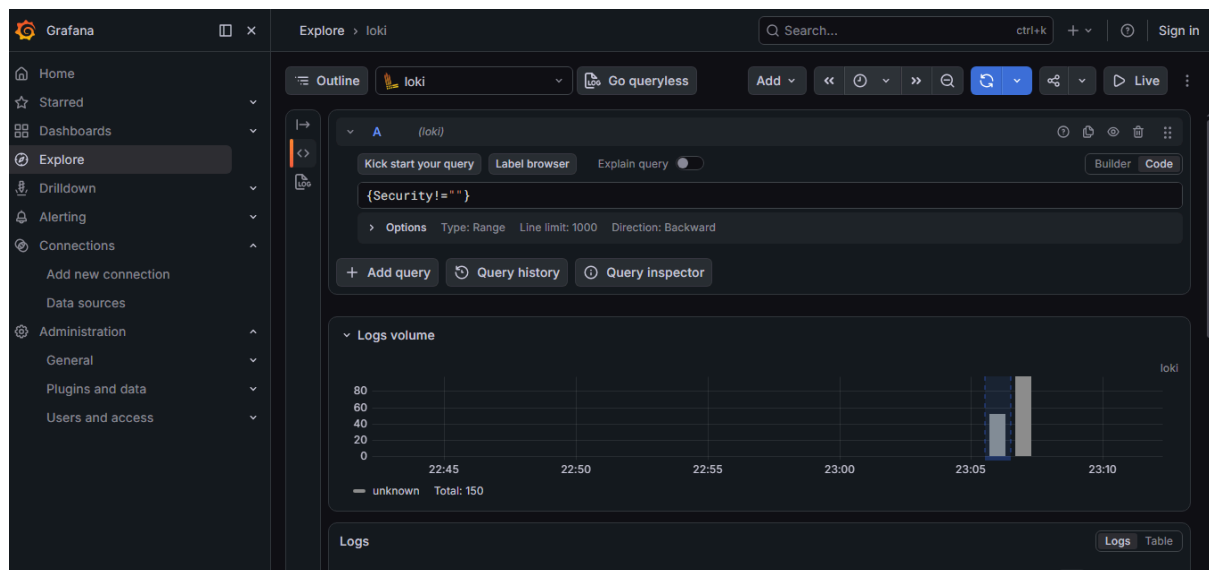


Figura 6. Consulta de eventos de seguridad.

Paso 4. Construcción de consultas específicas

Después de realizar las consultas básicas, se diseñaron consultas más específicas para localizar diferentes tipos de eventos registrados por VulnerableApp. Para ello se combinaron

las labels configuradas en Promtail con filtros de contenido de LogQL, permitiendo obtener información más precisa.

Consulta para localizar errores: {Application!=""} }

Esta consulta utiliza la label Application, la cual es asignada por Promtail cuando detecta palabras como Error o Exception dentro del registro. De esta forma únicamente se muestran los eventos relacionados con errores de la aplicación, facilitando su análisis sin necesidad de revisar todos los registros generados. Esto recupera: Errores de ejecución, excepciones registradas por la aplicación, eventos que requieren atención durante el monitoreo.

Consulta para localizar advertencias: {job="vulnerableapp"} |= "Warning" }

Para identificar advertencias se agregó el operador |= a la consulta general de VulnerableApp. Este operador permite buscar una palabra específica dentro del contenido del registro, por lo que únicamente se muestran los mensajes que contienen la palabra Warning. Esto recupera: Advertencias generadas por la aplicación, eventos que no representan un fallo crítico, pero que podrían requerir seguimiento.

Consulta para eventos de autenticación: {Security!=""} }

Esta consulta aprovecha la label Security, configurada previamente en Promtail para identificar automáticamente los registros relacionados con autenticación. Gracias a esta clasificación fue posible recuperar únicamente los eventos correspondientes a inicios de sesión y credenciales inválidas.

Posteriormente, la consulta se refinó utilizando un filtro adicional para localizar un usuario específico: {Security!=""} |= "hacker" }

Con esta modificación únicamente se mostraron los registros donde aparece el usuario hacker, permitiendo analizar con mayor facilidad los intentos de acceso fallidos realizados durante las pruebas. Con ello se recuperaron intentos de autenticación, credenciales inválidas, usuario involucrado, dirección IP desde la cual se realizó el intento, fecha y hora del evento.

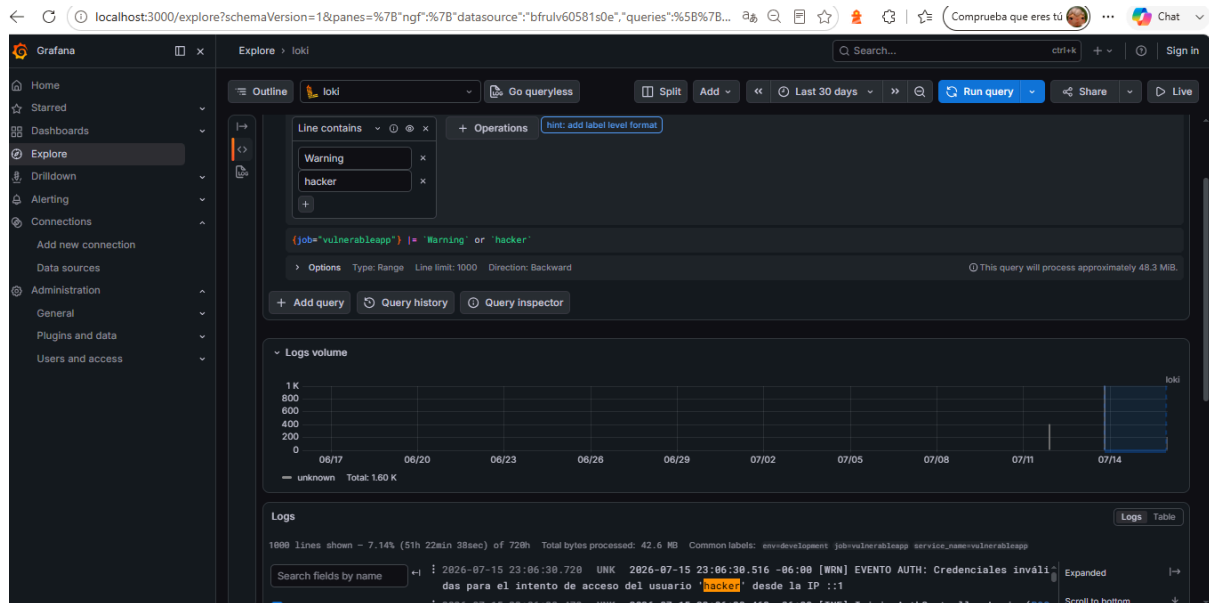


Figura 7. Consulta de advertencias combinada mediante labels y texto.

Paso 5. Generación de actividad

Con el objetivo de obtener suficientes registros para el análisis, se realizaron distintas acciones dentro de VulnerableApp.

Durante las pruebas se ejecutaron:

- inicios de sesión exitosos
- autenticaciones con credenciales incorrectas
- operaciones CRUD
- consultas al sistema
- generación de errores y excepciones controladas

Todas estas acciones fueron registradas automáticamente en Loki, permitiendo validar el correcto funcionamiento de las consultas realizadas.

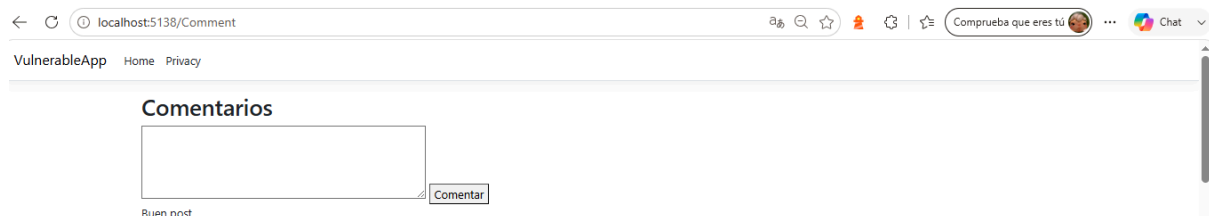


Figura 8. Generación de registros en VulnerableApp.

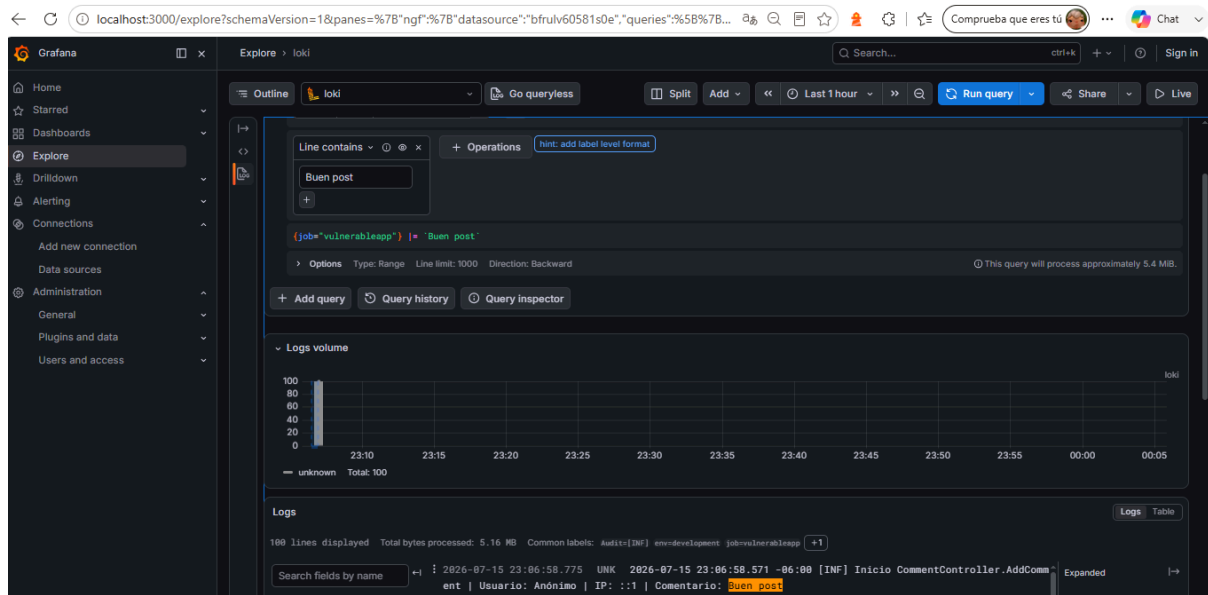


Figura 9. Nuevos registros recibidos en Grafana.

Paso 6. Análisis de un incidente

Para investigar el incremento de errores de autenticación se utilizó la consulta:

```
{Security!=""}{}
```

Esta consulta recuperó 98 registros de autenticación fallida dentro del rango de tiempo de las últimas 24 horas. El análisis de los eventos permitió identificar el momento de inicio del incidente, los usuarios involucrados y el comportamiento observado durante la prueba.

El primer registro observado fue:

2026-07-15 23:05:31.586 -06:00 [WRN] EVENTO AUTH: Credenciales inválidas para el intento de acceso del usuario 'admin' desde la IP ::1

Posteriormente se registró una gran cantidad de intentos fallidos del usuario hacker, con eventos consecutivos entre las 23:06:00 y las 23:06:30, lo que indica un comportamiento similar a una prueba de fuerza bruta o un ataque automatizado.

La evidencia muestra que:

- Los errores comenzaron a las 23:05:31.
- El evento fue clasificado automáticamente por Promtail mediante la label `Security="EVENTO AUTH"`.
- Los usuarios involucrados fueron admin y hacker.

Todos los intentos provinieron de la dirección IP ::1 (localhost), lo que confirma que la actividad fue generada desde el entorno local de pruebas.

Los registros fueron clasificados con nivel [WRN] (Warning), indicando autenticaciones inválidas pero sin provocar una caída de la aplicación.

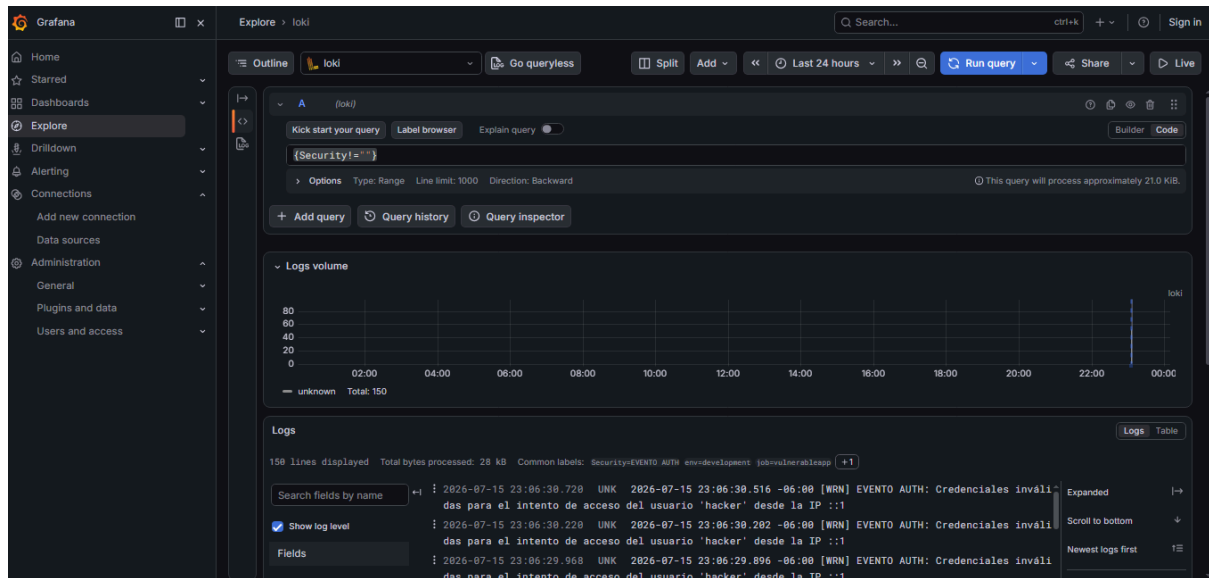


Figura 10. Investigación de autenticaciones fallidas mediante LogQL.

Paso 7. Construcción del dashboard

Finalmente se creó un dashboard en Grafana compuesto por tres paneles orientados al monitoreo de la aplicación.

1. Panel de Errores: Este panel muestra los registros clasificados como errores, permitiendo detectar rápidamente excepciones o fallos que puedan afectar el funcionamiento de la aplicación.
2. Panel de Autenticaciones: Presenta los eventos de seguridad relacionados con los intentos de inicio de sesión, facilitando la identificación de accesos exitosos y autenticaciones fallidas.
3. Panel de Auditoría: Muestra los registros informativos generados por la aplicación, permitiendo dar seguimiento a las acciones realizadas por los usuarios y mantener la trazabilidad del sistema.

La combinación de estos tres paneles proporciona una vista centralizada del estado de la aplicación y facilita el monitoreo continuo de los eventos más importantes.

En un ambiente de producción este dashboard puede utilizarse como una herramienta de monitoreo continuo. Los administradores pueden supervisar en tiempo real los intentos de autenticación, las acciones realizadas por los usuarios y los errores críticos registrados por la aplicación. Si alguno de los paneles presenta un incremento inesperado de eventos, es posible investigar rápidamente el origen del problema utilizando las consultas LogQL y tomar acciones antes de que el incidente afecte a más usuarios. De esta forma, el

dashboard contribuye a mejorar la seguridad, reducir los tiempos de respuesta y mantener una mayor visibilidad sobre el estado del sistema.

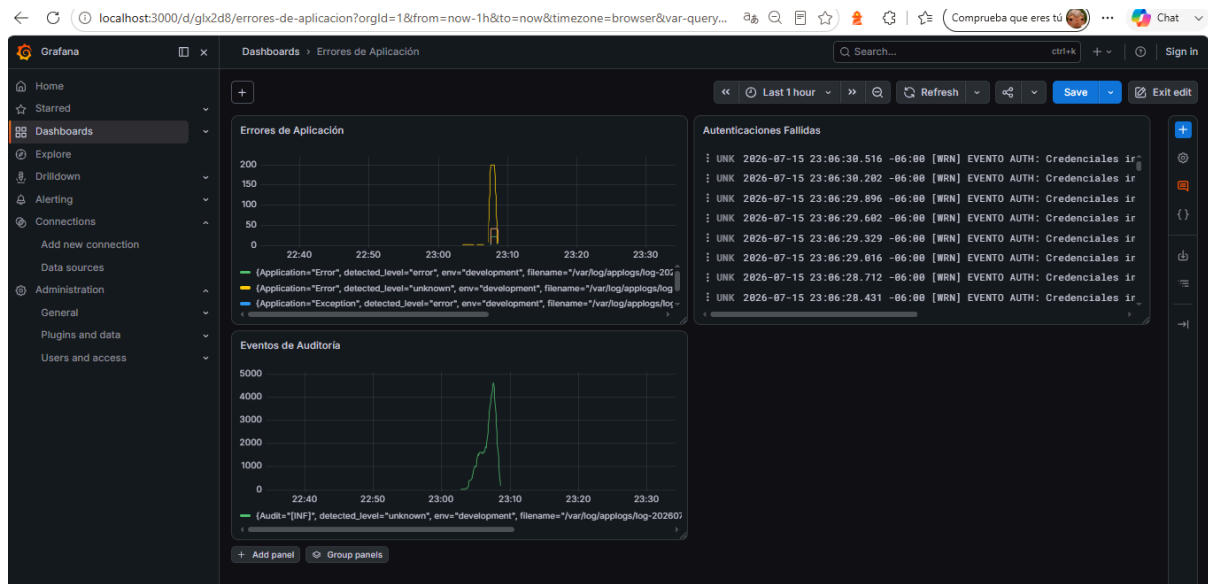


Figura 11. Dashboard de monitoreo de VulnerableApp.

Comparación entre Seq y Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda textual	Permite localizar registros mediante búsquedas sobre las propiedades estructuradas enviadas por Serilog.	Permite realizar búsquedas utilizando LogQL, combinando labels y filtros por contenido.
Filtros por labels	Trabaja principalmente con las propiedades registradas por Serilog.	Utiliza labels para localizar rápidamente grupos específicos de registros.
Dashboards	Cuenta con opciones básicas de visualización.	Permite crear dashboards totalmente personalizables para monitoreo continuo.
Investigación de incidentes	Facilita revisar eventos individuales con gran nivel de detalle.	Permite correlacionar eventos y analizar tendencias mediante consultas y paneles interactivos.
Uso principal	Análisis detallado de registros durante el desarrollo y depuración.	Observabilidad, monitoreo y análisis continuo de aplicaciones en producción.

Caso de estudio

Situación: Se reportó un incremento de errores de autenticación en VulnerableApp. Utilizando únicamente Grafana y LogQL se realizó la investigación de los registros almacenados en Loki.

¿Cuándo comenzaron?

Los primeros errores comenzaron el 15 de julio de 2026 a las 23:05:31, de acuerdo con el primer evento recuperado por la consulta `{Security!=""}`.

¿Qué módulo los generó?

Los registros fueron clasificados por Promtail como eventos de autenticación mediante la label `Security="EVENTO AUTH"`, correspondiente al flujo de validación de credenciales de la aplicación.

¿Qué usuarios estuvieron involucrados?

Se identificaron intentos fallidos para los usuarios:

admin, hacker

¿Qué excepción se registró?

No se registró una excepción crítica de la aplicación; los eventos corresponden a advertencias de autenticación inválida con nivel [WRN], indicando rechazo de credenciales sin afectar la disponibilidad del sistema.

¿Qué evidencia respalda las conclusiones?

La evidencia se obtuvo mediante la consulta `{Security!=""}`, donde se observaron los registros con marcas de tiempo, usuarios involucrados y la dirección IP de origen ::1. Además, Grafana reportó un total de 98 eventos en las últimas 24 horas, confirmando el incremento de intentos de autenticación fallida.

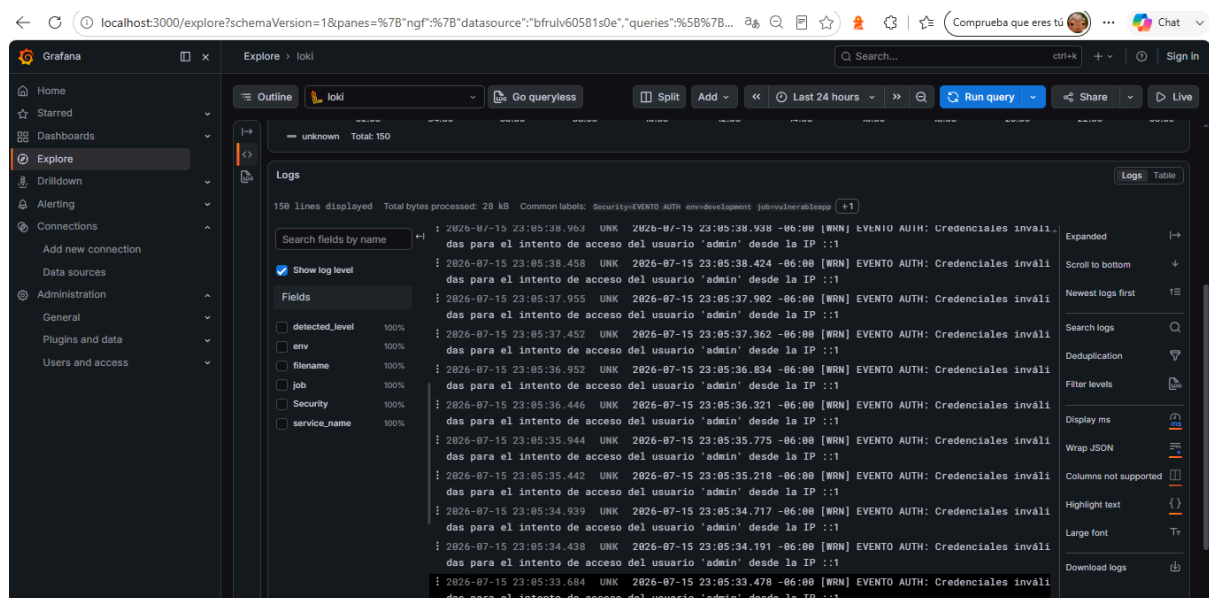


Figura 12. Evidencia del incremento de autenticaciones fallidas detectado con LogQL.

Pregunta de reflexión: ¿Qué ventajas aporta LogQL frente a revisar directamente archivos de texto o realizar búsquedas simples en Seq?

LogQL ofrece una forma mucho más eficiente de analizar registros que revisar manualmente archivos de texto. Cuando una aplicación genera cientos o miles de eventos, localizar un error específico en un archivo puede resultar tardado. En cambio, LogQL permite filtrar la información mediante labels, contenido del mensaje e intervalos de tiempo, obteniendo únicamente los registros relevantes en cuestión de segundos.

Aunque Seq también permite realizar búsquedas sobre los registros estructurados enviados por Serilog, Grafana junto con LogQL ofrece una mayor flexibilidad para combinar filtros, reutilizar consultas y construir dashboards que muestran el comportamiento del sistema en tiempo real. Esto facilita el monitoreo continuo, la detección de incidentes y el análisis de tendencias sin necesidad de revisar los registros uno por uno.

Conclusiones

Durante esta práctica se comprobó que LogQL es una herramienta muy útil para consultar y analizar los registros almacenados en Loki de manera rápida y organizada. Gracias al uso de labels configuradas en Promtail fue posible filtrar eventos de autenticación, auditoría y errores sin necesidad de revisar manualmente grandes cantidades de información.

Las consultas desarrolladas permitieron localizar eventos específicos, analizar un incidente de autenticación y obtener información relevante como la fecha de inicio, los usuarios involucrados y el tipo de evento registrado. Esto demostró que una correcta clasificación de los registros facilita considerablemente las tareas de monitoreo e investigación.

Asimismo, la construcción del dashboard en Grafana permitió centralizar la información más importante de la aplicación en una sola vista. Los paneles de autenticaciones, auditoría y errores críticos ofrecen una supervisión continua del sistema y facilitan la detección temprana de comportamientos anómalos.

Finalmente, la práctica permitió comprender que Grafana y Loki complementan el trabajo realizado con Seq. Mientras Seq resulta muy útil para inspeccionar registros individuales durante el desarrollo, Grafana junto con LogQL proporciona mayores capacidades para el monitoreo continuo, la correlación de eventos y el análisis de incidentes en entornos de producción, convirtiéndose en una solución más completa para la observabilidad de aplicaciones.